



北京航空航天大学
BEIHANG UNIVERSITY

机器人导航python实践(入门)

Lecture8-寻迹

北航 国新院 实验实践课
智能系统与人形机器人国际研究中心



教师： 欧 阳 老 师



邮 箱： ouyangkid@buaa.edu.cn



学 期： 2025年秋季

目录

Contents

01 课程内容安排

02 Pure pursuit tracking

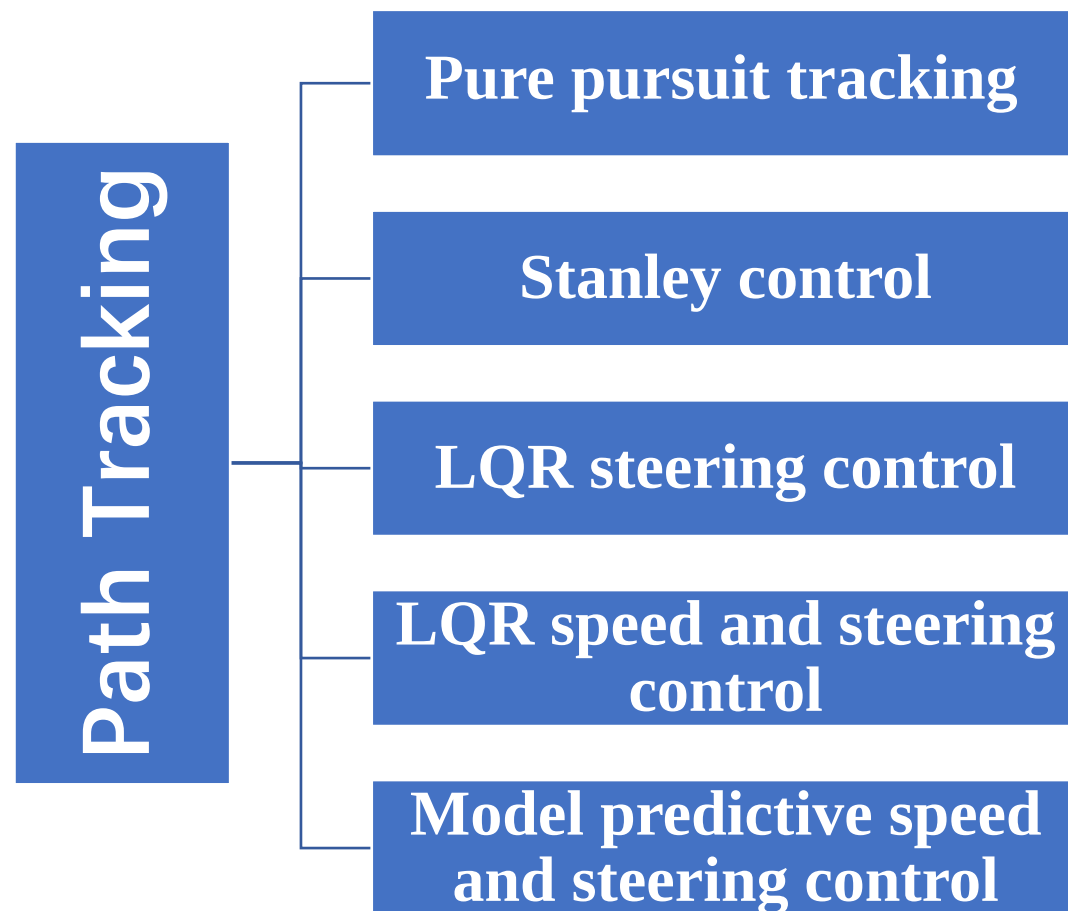
03 Stanley control

04 LQR steering control

05 LQR speed and steering control

06 Model predictive speed and steering control

» 内容安排



路径跟踪/寻迹 (Path Tracking) 指机器人在跟踪路径规划器生成的参考路径的同时，满足自身运动约束的能力；控制器需考虑建模误差及其它的不确定性，通常分为反馈控制和优化控制技术。最新方案还包括基于深度学习的智能控制。

目录

Contents

01 课程内容安排

02 **Pure pursuit tracking**

03 Stanley control

04 LQR steering control

05 LQR speed and steering control

06 Model predictive speed and steering control

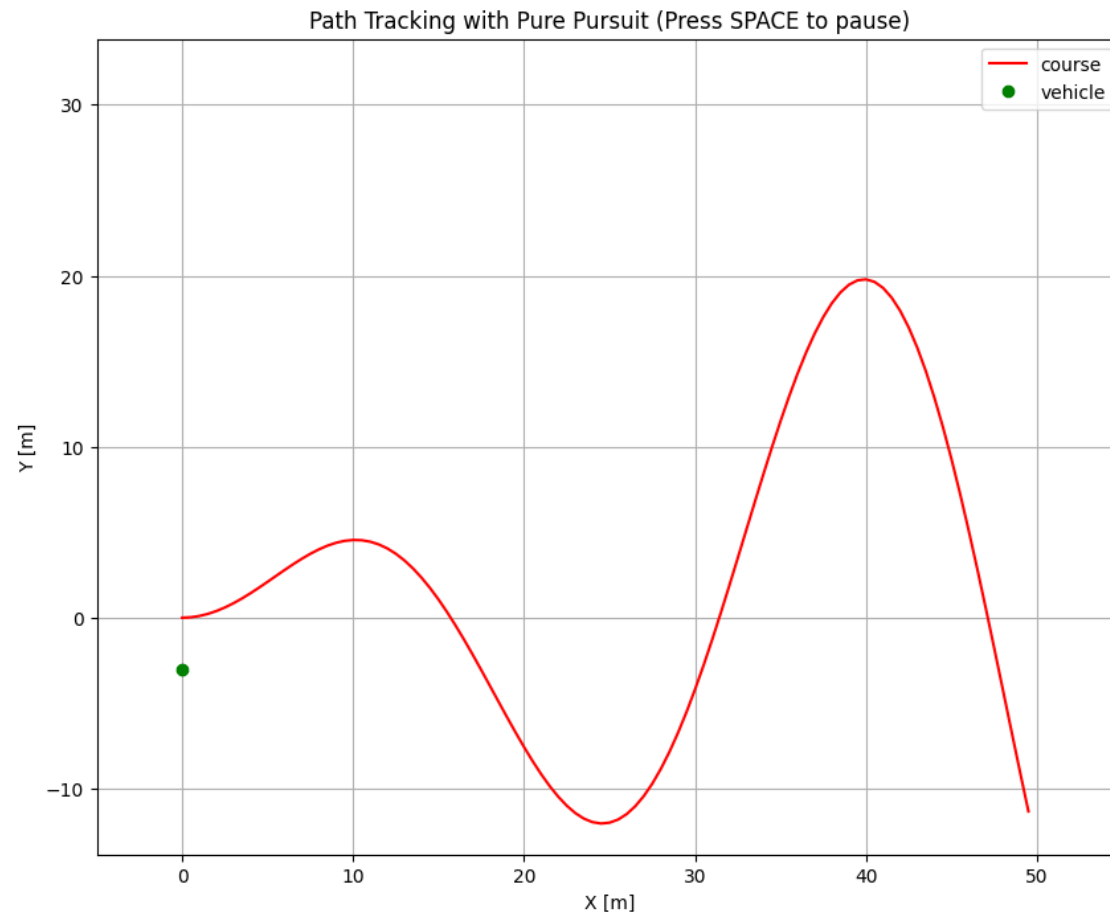
Part 2 | Pure pursuit tracking

寻迹/路径跟踪

- 定位：粒子滤波算法
- 建图：光线投射网格地图
- 路径规划：基于网格地图的Dijkstra
- **寻迹：纯追踪跟踪 (Pure Tracking)**
 - 点击运行按钮，matplotlib绘制并实时更新算法结果。
 - **红色：**规划轨迹。
 - **绿色X：**目标点
 - **蓝色：**控制车辆 + 实际轨迹。

前面的示例，为了突出对应模块的算法本身，对Agent的运动进行简化，仅考虑质点的运动坐标、速度和航向+控制噪声。

但寻迹任务中，运动体的动力学和运动学模型，是限制其**运动能力**的最大要素，需要进行建模。

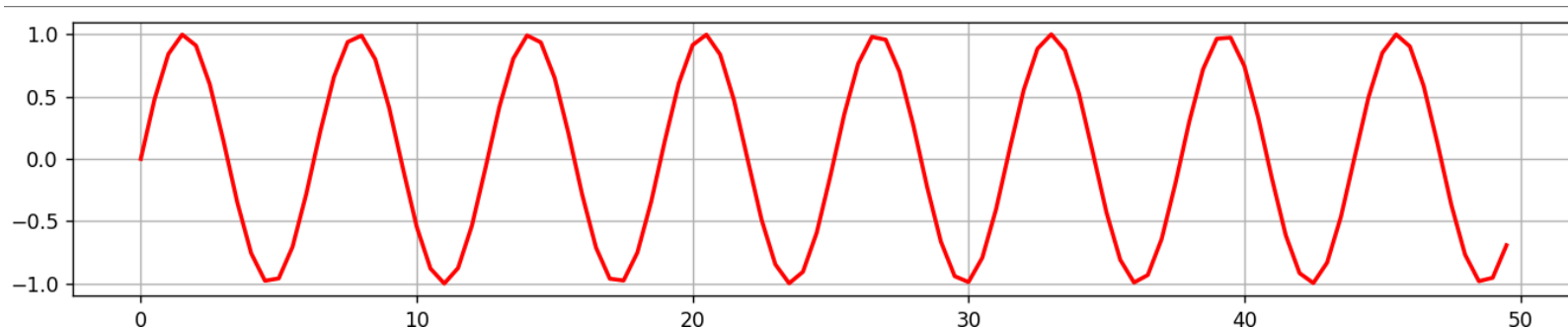
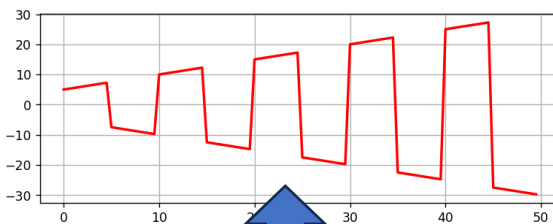


Part 2 | Pure pursuit tracking



寻迹/路径跟踪

- 目标轨迹：定义一条从原点出发的正弦曲线。



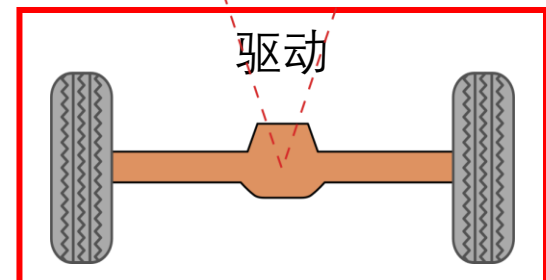
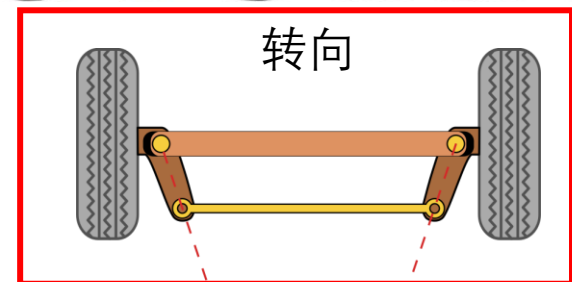
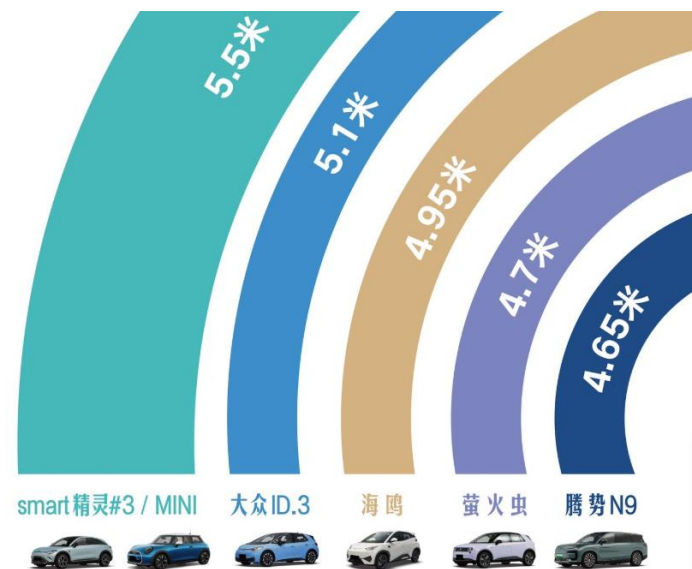
```
cx = -1 * np.arange(0, 50, 0.5) if is_reverse_mode else np.arange(0, 50, 0.5)
cy = [math.sin(ix) for ix in cx]
```

理想情况下：希望车辆能够从起点沿轨迹移动到终点。

实际情况：车辆受自身动力学因素限制，其行动能力并不是无限的。除了速度以外，车辆的尺寸，会影响车的运动学属性，进而限制任意运动。

比较著名的有：自行车模型（2轮），三轮模型和四轮模型。
考虑的轮越多，计算公式越复杂，估计越精准。

越大”得房率”越高
转向半径越小越灵活



Part 2 | Pure pursuit tracking

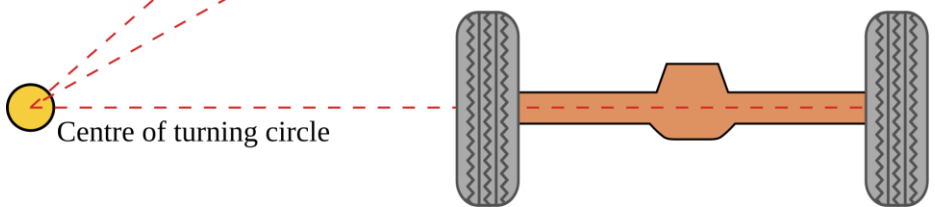
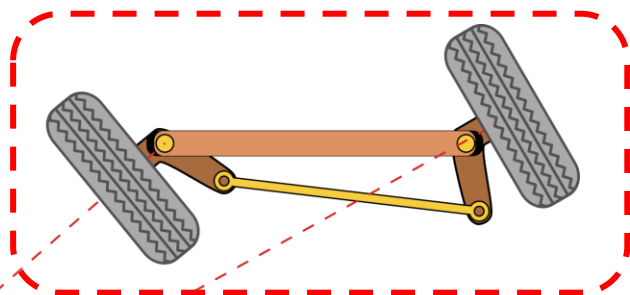


寻迹/路径跟踪

复杂
精细

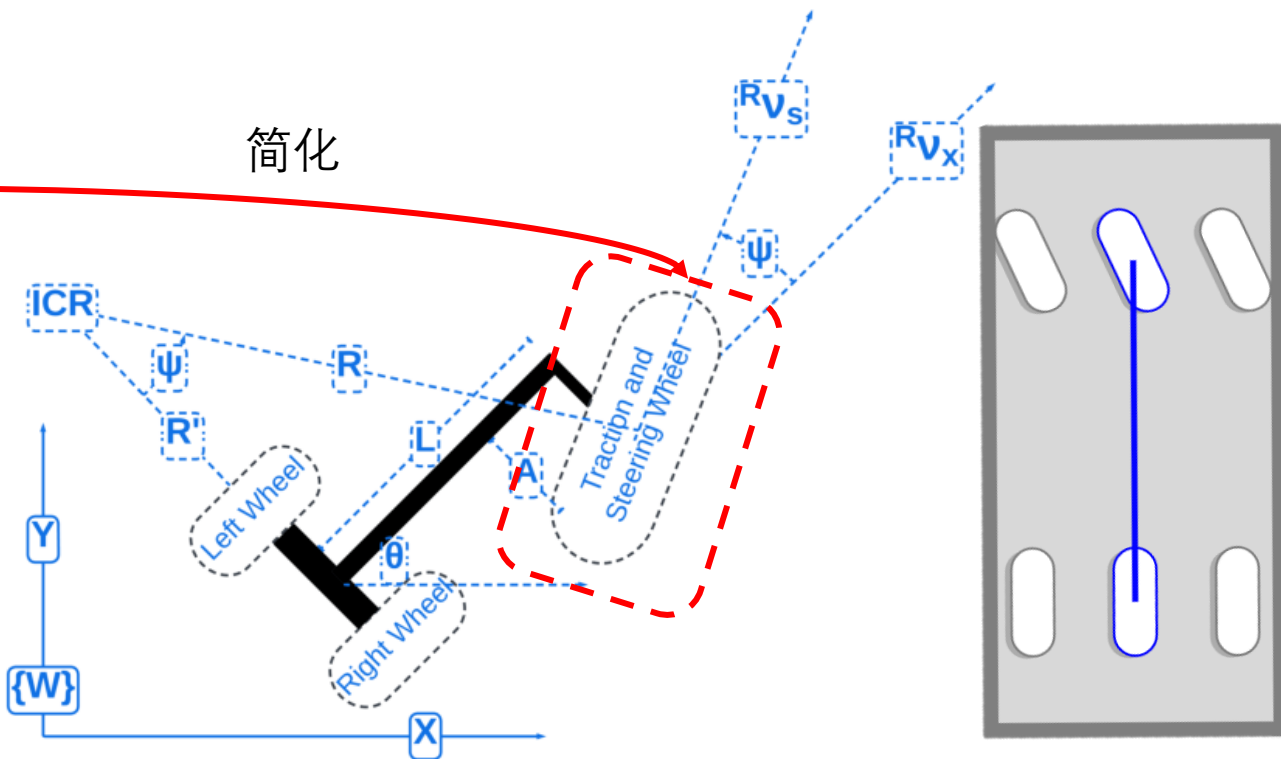
简单
粗

- 车辆运动学模型



阿克曼四轮模型中，两个前轮的转向半径不同
内大外小，最接近真实情况。

简化



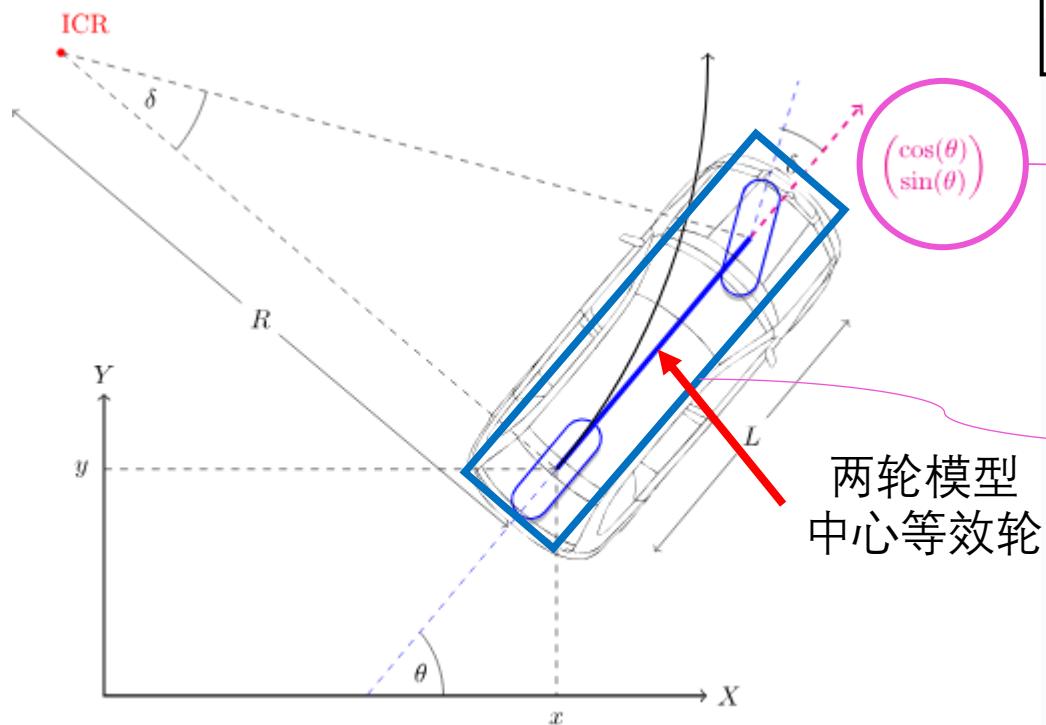
简化四轮模型中的横向控制，
就是三轮车模型

进一步简化后轮间距
得到自行车模型
非常接近之前的质点模型

Part 2 | Pure pursuit tracking



寻迹/路径跟踪



每个时间片下的车辆运动状态存入States列表

```
class State:
```

```
def __init__(self, x=0.0, y=0.0, yaw=0.0, v=0.0, is_reverse=False):
```

```
self.x = x
```

```
self.y = y
```

```
self.yaw = yaw
```

```
self.v = v
```

```
self.direction = -1 if is_reverse else 1 # Direction based on reverse flag
```

```
self.rear_x = self.x - self.direction * ((WB / 2) * math.cos(self.yaw))
```

```
self.rear_y = self.y - self.direction * ((WB / 2) * math.sin(self.yaw))
```

```
def update(self, a, delta):
```

```
self.x += self.v * math.cos(self.yaw) * dt
```

```
self.y += self.v * math.sin(self.yaw) * dt
```

位置更新

```
self.yaw += self.direction * self.v / WB * math.tan(delta) * dt
```

速度更新

```
self.yaw = angle_mod(self.yaw)
```

航向角更新

```
self.v += a * dt
```

```
self.rear_x = self.x - self.direction * ((WB / 2) * math.cos(self.yaw))
```

```
self.rear_y = self.y - self.direction * ((WB / 2) * math.sin(self.yaw))
```

轴距L=2.9m

车长=3.9m # 前后个加0.5

车宽=2m

车轮直径=0.6

车轮宽度=0.2

Part 2 | Pure pursuit tracking

寻迹/路径跟踪

车辆控制:

- 横向=转向角度 # 航向角的变化
- 纵向=加速度 # 速度的变化

设目标速度10km/h=10/3.6 (m/s)

车辆的运动学特性建模:

- 最大转向半径 $\delta = \pi/4$ # 车轮的最大转向角度
- 速度比例增益系数 $K_p=1.0$ # P控制器

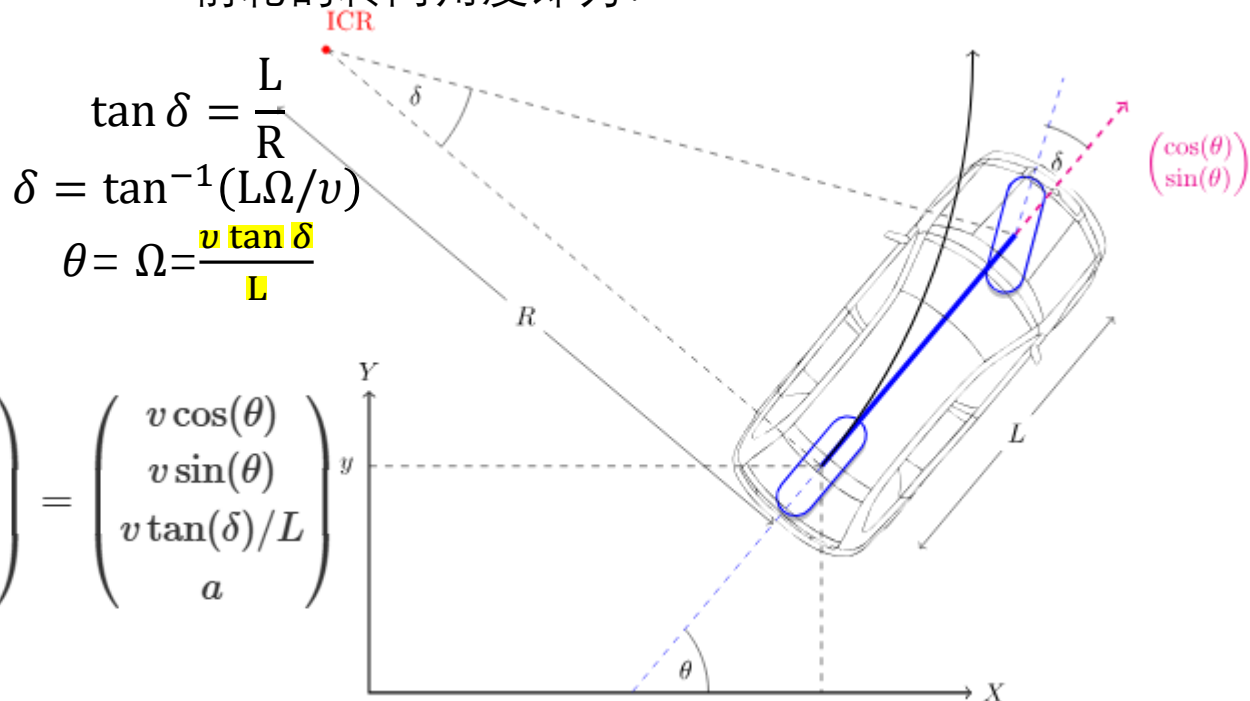
对于加速度, 使用简单的P控制器 (比例控制器), 基于当前速度和目标速度的差进行更新, 并且假设执行率100%, 调节 K_p 可以模拟比例增益 (实际为PID控制的简化)。

```
def proportional_control(target, current):  
    a = Kp * (target - current)  
    return a
```

$A = \pi r^2$ 其他类型控制器可以进行更复杂和精确的测量。

对于航向角 (仅考虑XOY平面), 刚体的车辆存在一个瞬时转动中心ICR, 一般用来作为车辆运动分析的参考点。

以两轮车轮模型为例, 车辆实际与以瞬时转动中心为圆, 后轮中心为切点, 进行切向运动。前轮的转向角度即为:



$$\tan \delta = \frac{L}{R}$$
$$\delta = \tan^{-1}(L\Omega/v)$$
$$\theta = \Omega = \frac{v \tan \delta}{L}$$

$$\frac{d}{dt} \begin{pmatrix} x \\ y \\ \theta \\ v \end{pmatrix} = \begin{pmatrix} v \cos(\theta) \\ v \sin(\theta) \\ v \tan(\delta)/L \\ a \end{pmatrix}$$

*前轮以角速度绕后轮旋转

Part 2 | Pure pursuit tracking



寻迹/路径跟踪

- Pure Tracking纯追踪

在纯追踪法中，会在目标轨迹上确定一个目标点(TP)，目标点为车辆到轨迹最近的点。
基于目标点和预瞄距离(Lfc)，搜索目标轨迹上的预瞄点。

搜索车辆中心到目标轨迹最小距离点

通过速度和增益动态更新 预瞄距离
高速的时候看得更远，转向半径平缓
低速看的相对近，对路径变化敏感
防止震荡、超调

更新最近点

基于速度更新预瞄距离

```
Lf = k * state.v + Lfc # update look ahead distance
```

沿路径查找超过预瞄距离 Lfc的预瞄点

```
while Lf > state.calc_distance(self.cx[ind], self.cy[ind]):  
    if (ind + 1) >= len(self.cx):  
        break # not exceed goal  
    ind += 1
```

```
def search_target_index(self, state):  
    if self.old_nearest_point_index is None:  
        dx = [state.rear_x - icx for icx in self.cx]  
        dy = [state.rear_y - icy for icy in self.cy]  
        d = np.hypot(dx, dy)  
        ind = np.argmin(d)  
        self.old_nearest_point_index = ind  
    else:  
        ind = self.old_nearest_point_index  
        distance_this_index = state.calc_distance(self.cx[ind],  
                                                    self.cy[ind])  
  
        while True:  
            distance_next_index = state.calc_distance(self.cx[ind + 1],  
                                                        self.cy[ind + 1])  
  
            if distance_this_index < distance_next_index:  
                break  
  
            ind = ind + 1 if (ind + 1) < len(self.cx) else ind  
            distance_this_index = distance_next_index  
        self.old_nearest_point_index = ind
```

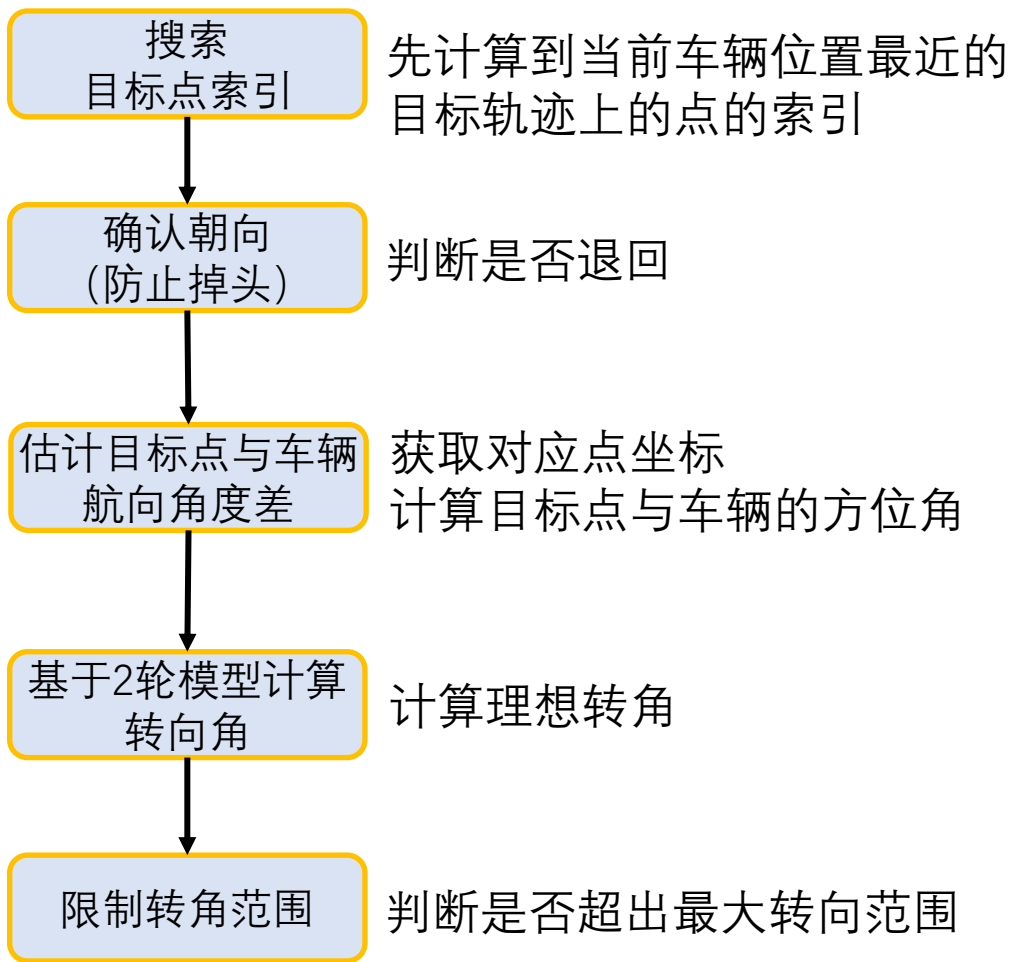
Part 2 | Pure pursuit tracking

Python环境



寻迹/路径跟踪

- 横向控制



```
def pure_pursuit_steer_control(state, trajectory, pind):
    ind, Lf = trajectory.search_target_index(state)

    if pind >= ind:
        ind = pind

    if ind < len(trajectory.cx):
        tx = trajectory.cx[ind]
        ty = trajectory.cy[ind]
    else:
        tx = trajectory.cx[-1]
        ty = trajectory.cy[-1]
        ind = len(trajectory.cx) - 1

    alpha = math.atan2(ty - state.rear_y, tx - state.rear_x) - state.yaw

    delta = state.direction * math.atan2(2.0 * WB * math.sin(alpha) / Lf, 1.0)

    delta = np.clip(delta, -MAX_STEER, MAX_STEER)

    return delta, ind
```


Part 2 | Pure pursuit tracking

寻迹/路径跟踪

总体流程

PP的横向仅考虑（动力学约束下的）
当前转角和目标点之间的相对关系。
纵向则受最大速度限制。

初始化目标轨迹

初始车辆状态

搜索目标点

车辆控制指令生成

状态更新

判断终点

车辆信息

目标速度

速度控制

角度控制

刹车/油门

比例控制P

方向盘

纯追踪



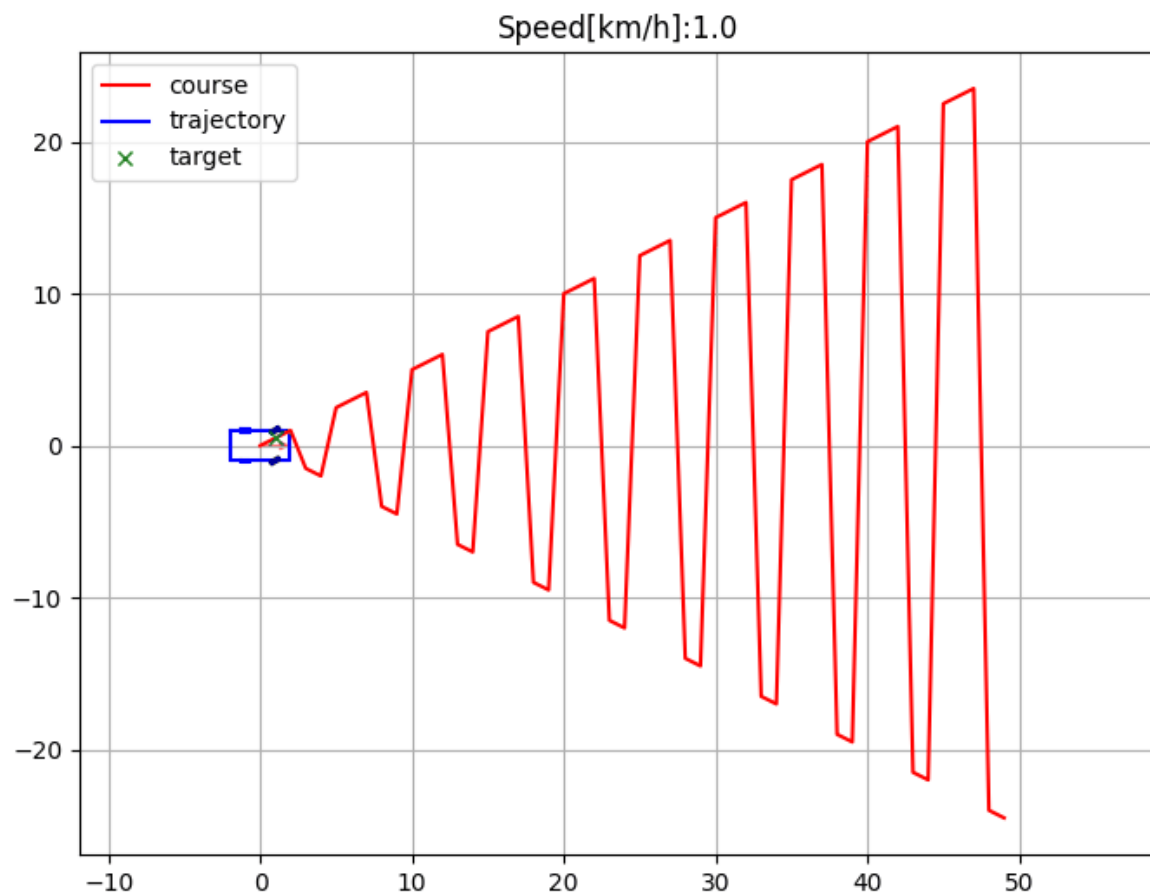
实际上pure tracking在整个



Part 2 | Pure pursuit tracking



寻迹/路径跟踪



如果将轨迹换成一个更复杂的秋名山“连环发卡弯”，PP虽然也能够实现路径追踪，但是规划的轨迹基本上需要翻越山头路，形成了类似拖拉机种田时需要的“梨形”轨迹。因此，PP不适合转角变化较大的复杂轨迹追踪。

目录

Contents

01 课程内容安排

02 Pure pursuit tracking

03 **Stanley control**

04 LQR steering control

05 LQR speed and steering control

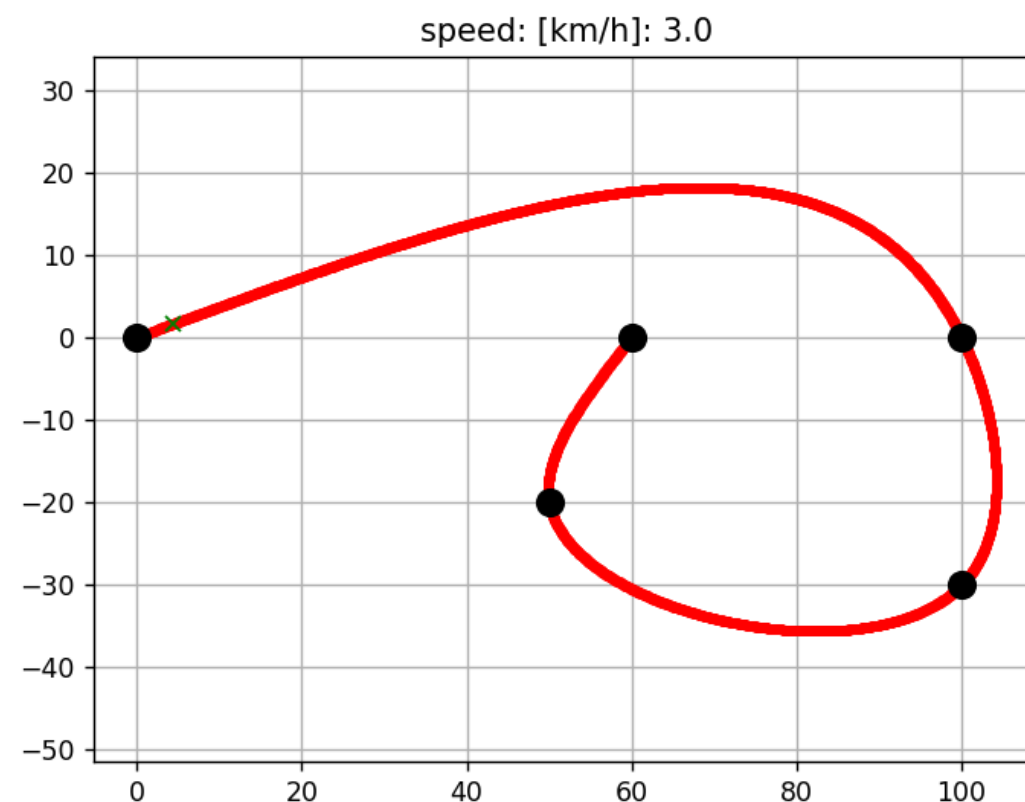
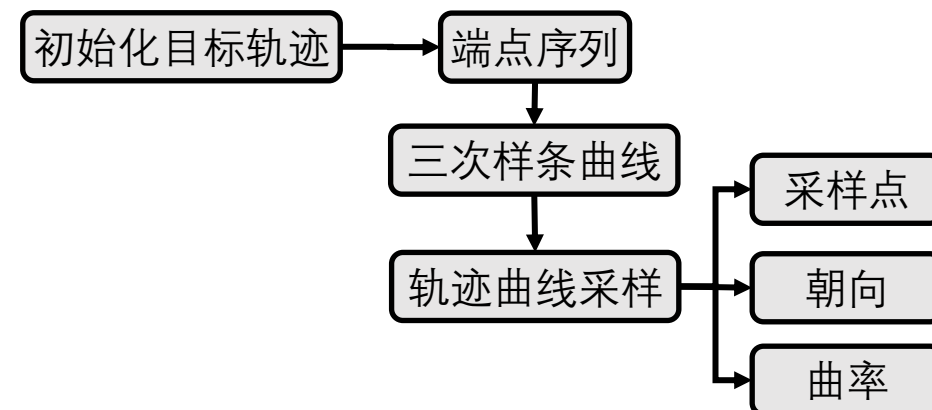
06 Model predictive speed and steering control

Part 3 | Stanley control

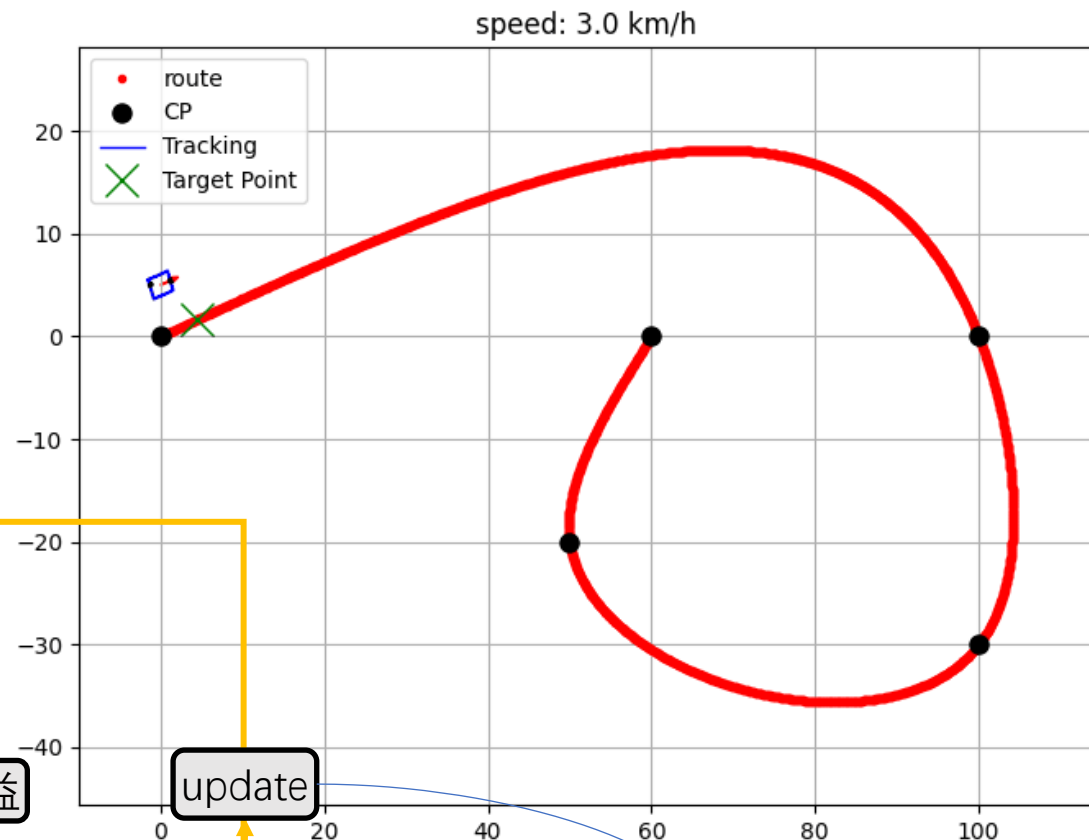
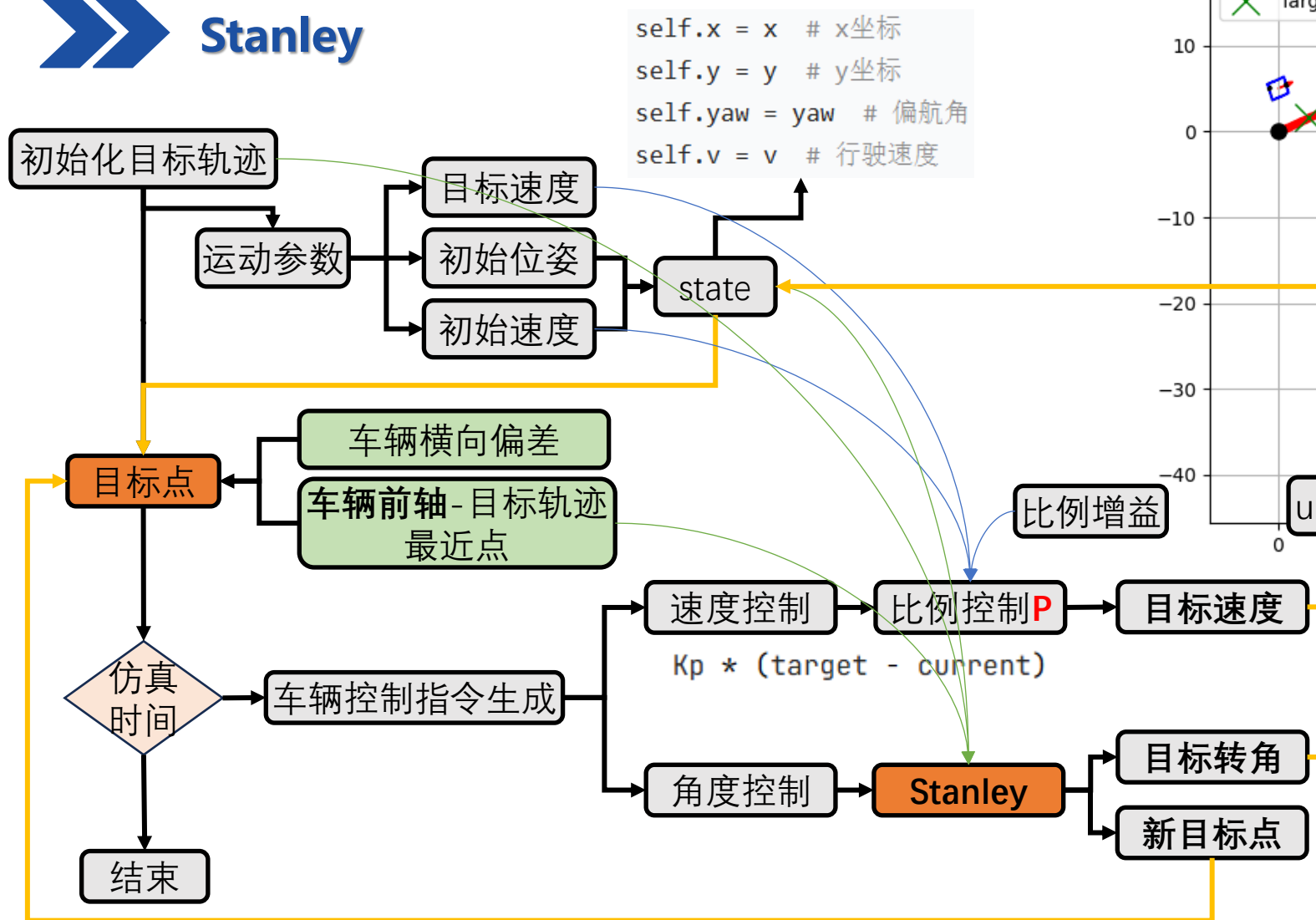


Stanley是斯坦福参加DARPA在05年无人驾驶挑战赛的车型。

车辆	团队名称	团队主场	比赛用时 (h:m)	排名
Stanley	Stanford Racing Team	斯坦福大学, Palo Alto, California	6:54	第一
Sandstorm	Red Team	卡内基梅隆大学, Pittsburgh, Pennsylvania	7:05	第二
H1ghlander	Red Team Too		7:14	第三
Kat-5	Team Gray	格雷保险公司, Metairie, Louisiana	7:30	第四
TerraMax	Team TerraMax	奥什科什卡车公司, Oshkosh, Wisconsin	12:51	超时



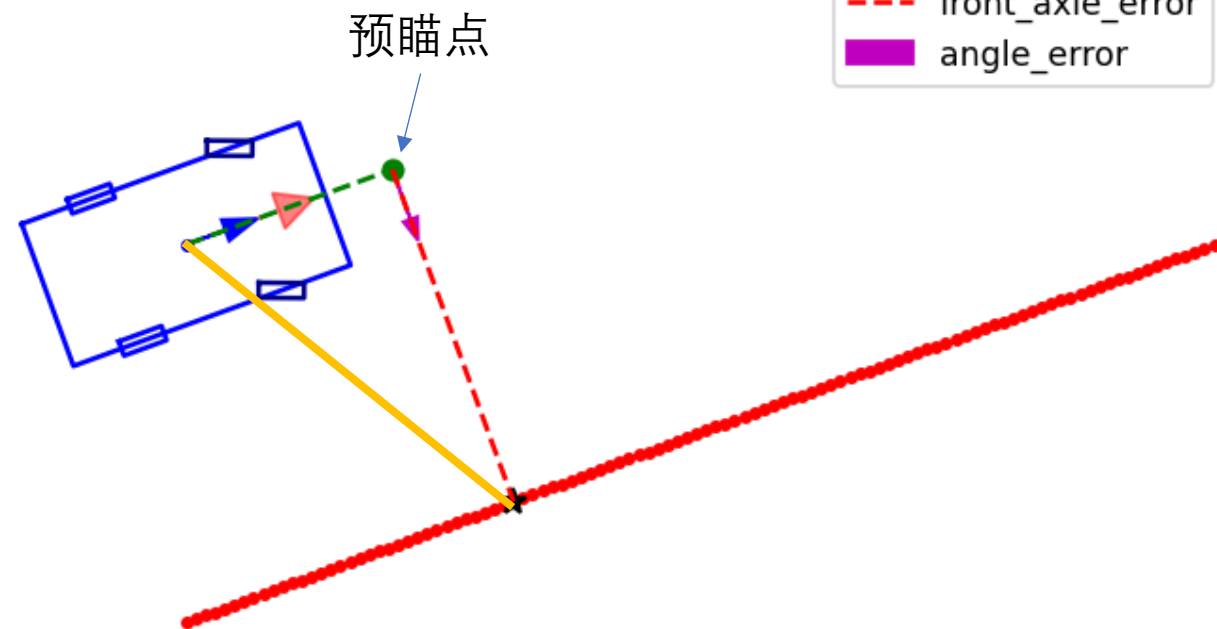
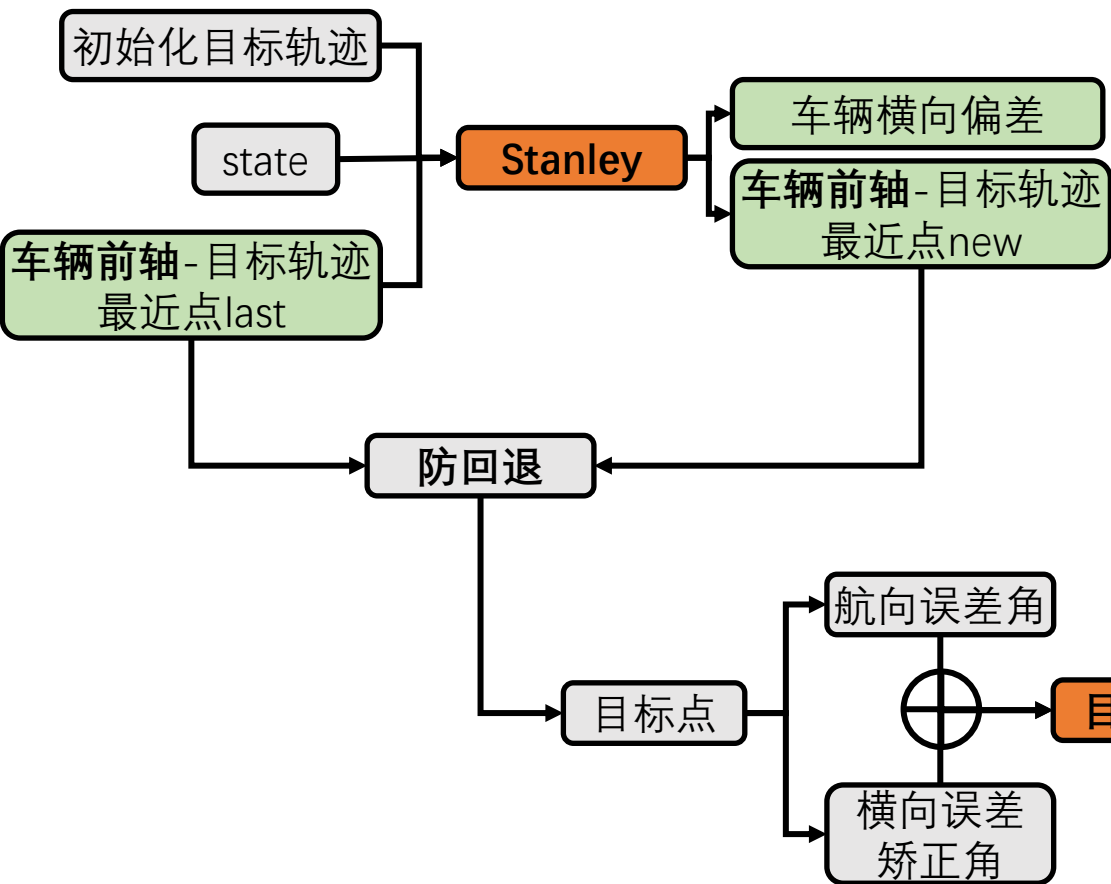
Part 3 | Stanley control



```
# 限制转向角在最大范围内（防止过度转向）
delta = np.clip(delta, -max_steer, max_steer)
```

Stanley控制算法仅依赖目标点进行调控，通过目标点的 **横向误差** 和 **航向误差** 进行矫正，实现精确路径跟踪。适合大曲率，频繁需要进行大角度变化的路径，如泊车、S路、U-turn等

Part 3 | Stanley control



车辆当前航向 与 路径上目标点对应航向（三次样条插值）的差。
矫正车辆朝向：希望车辆到目标点时，能够保持曲率一致的角度。

以车辆当前的速度（省略时间）作为向前的距离（绿色虚线），
以及横向偏差（红色虚线）构建的直角三角形的夹角。
缩小 预瞄点—目标点 之间的差

目录

Contents

01 课程内容安排

02 Pure pursuit tracking

03 Stanley control

04 **LQR steering control**

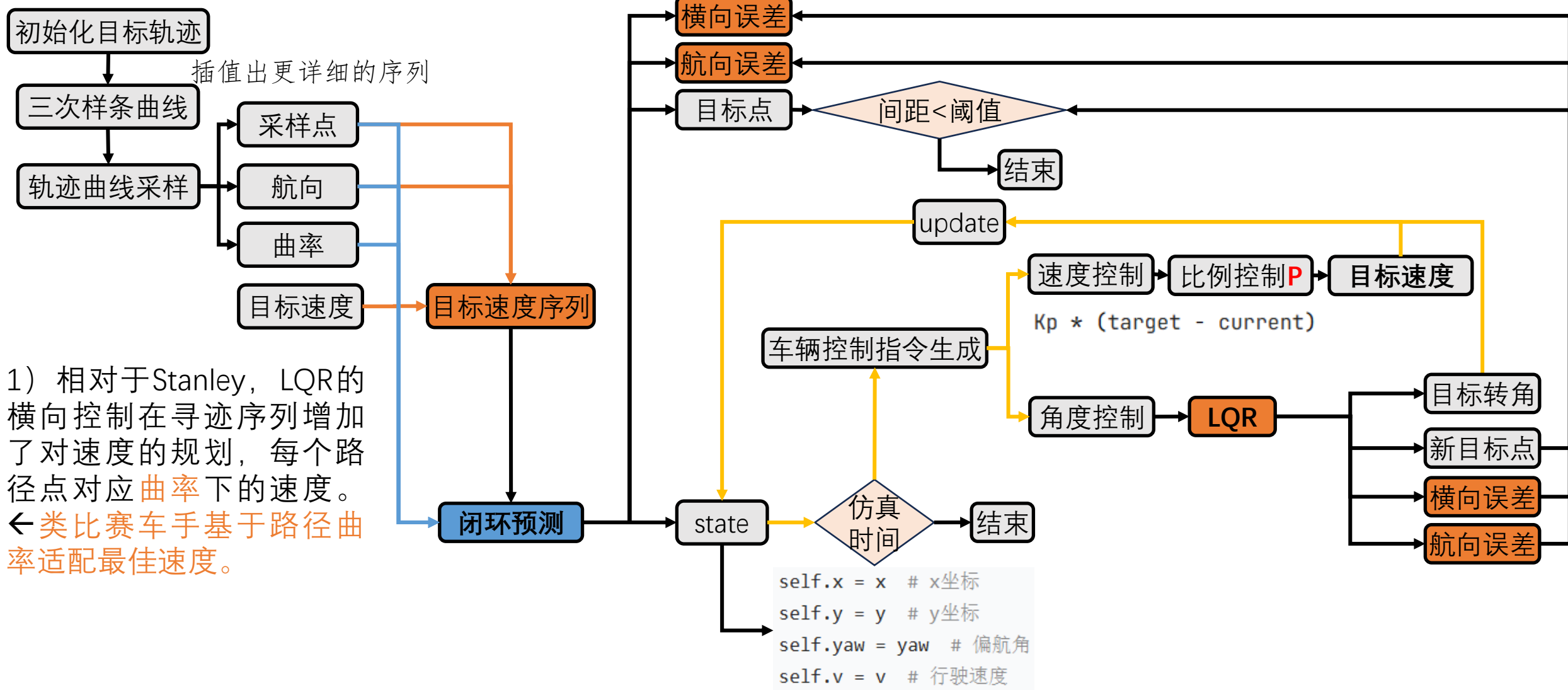
05 LQR speed and steering control

06 Model predictive speed and steering control

Part 4 | LQR steering control

LQR steering control

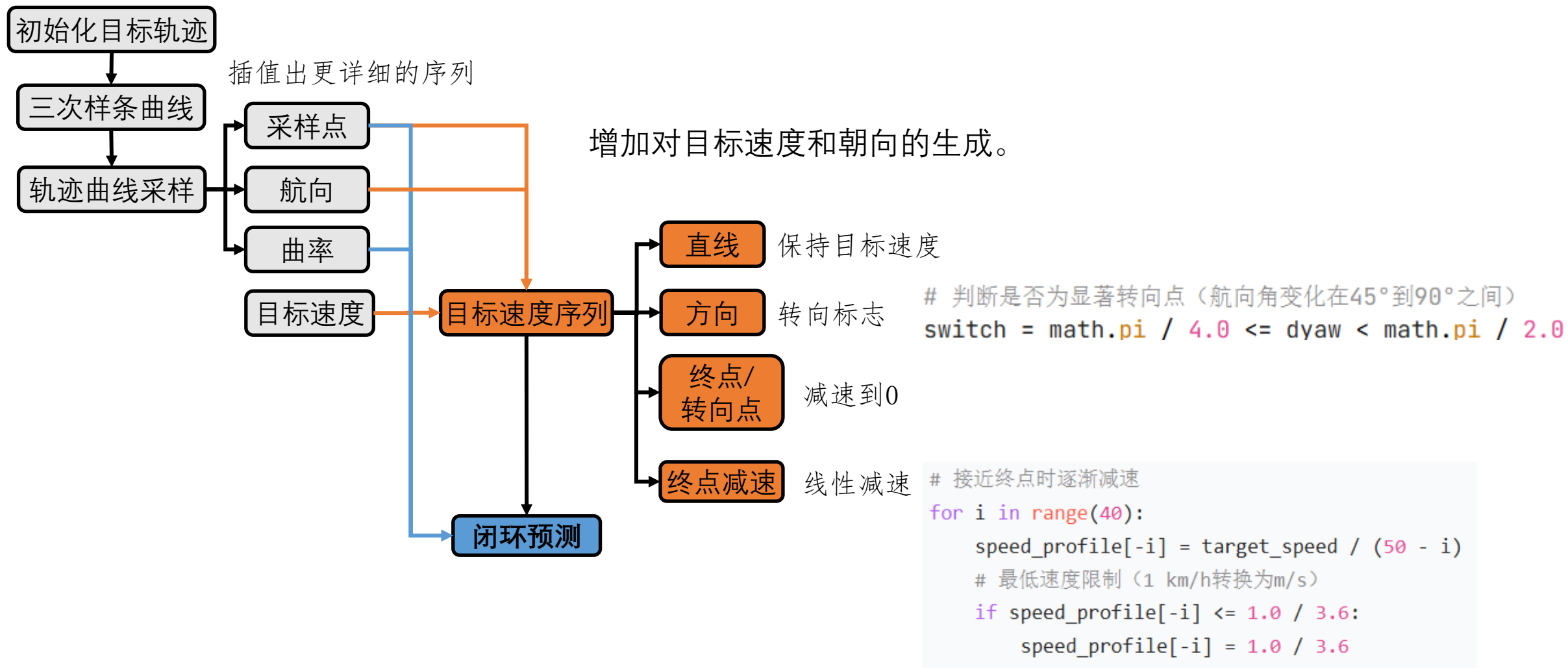
LQR (Linear Quadratic Regulator, 线性二次型调节器) 是一种转向控制器设计方法, 核心思想是通过最小化“跟踪误差”与“控制量消耗”的加权组合, 实现车辆的精准横向控制。自动驾驶的车道保持、路径跟踪等场景中应用广泛, 能在“控制精度”和“控制平稳性”之间找到最优平衡。



Part 4 | LQR steering control



LQR steering control



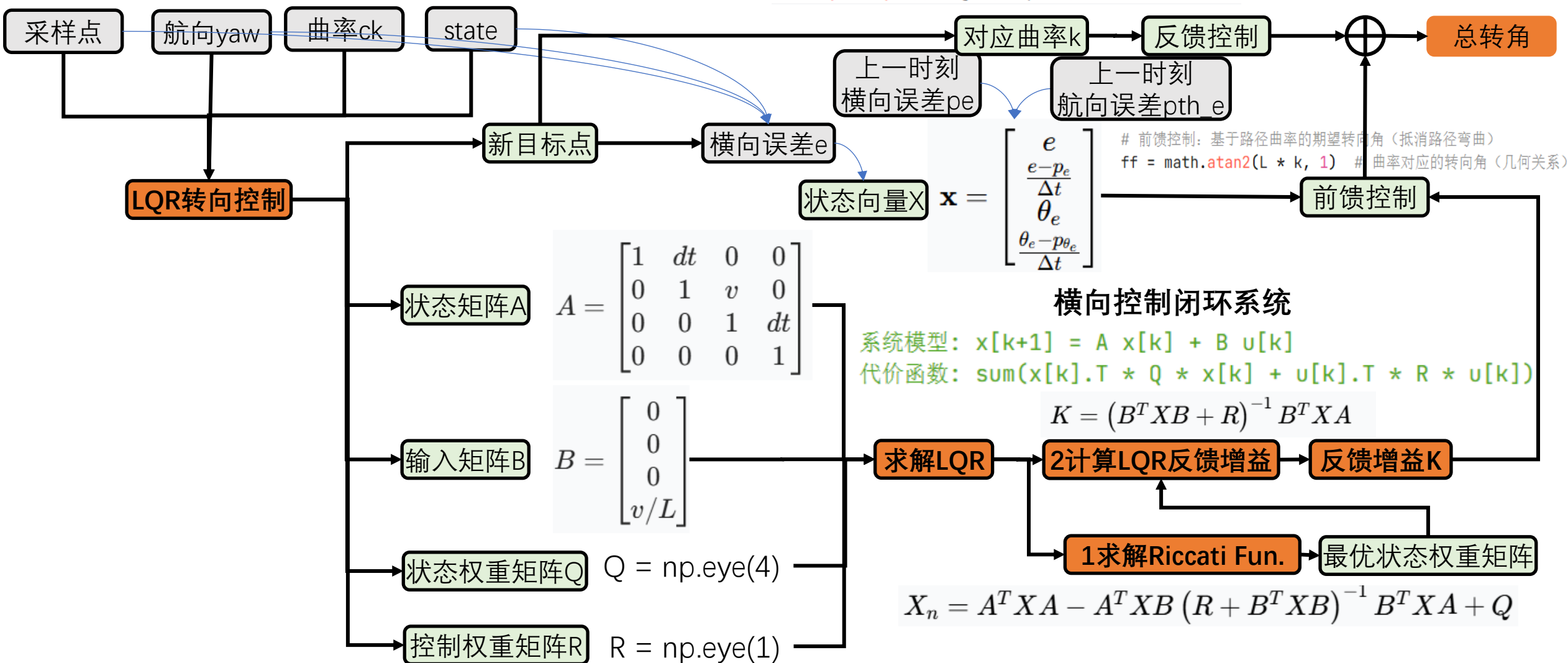
Part 4 | LQR steering control



LQR steering control

反馈控制：基于LQR的误差修正（抵消跟踪偏差）

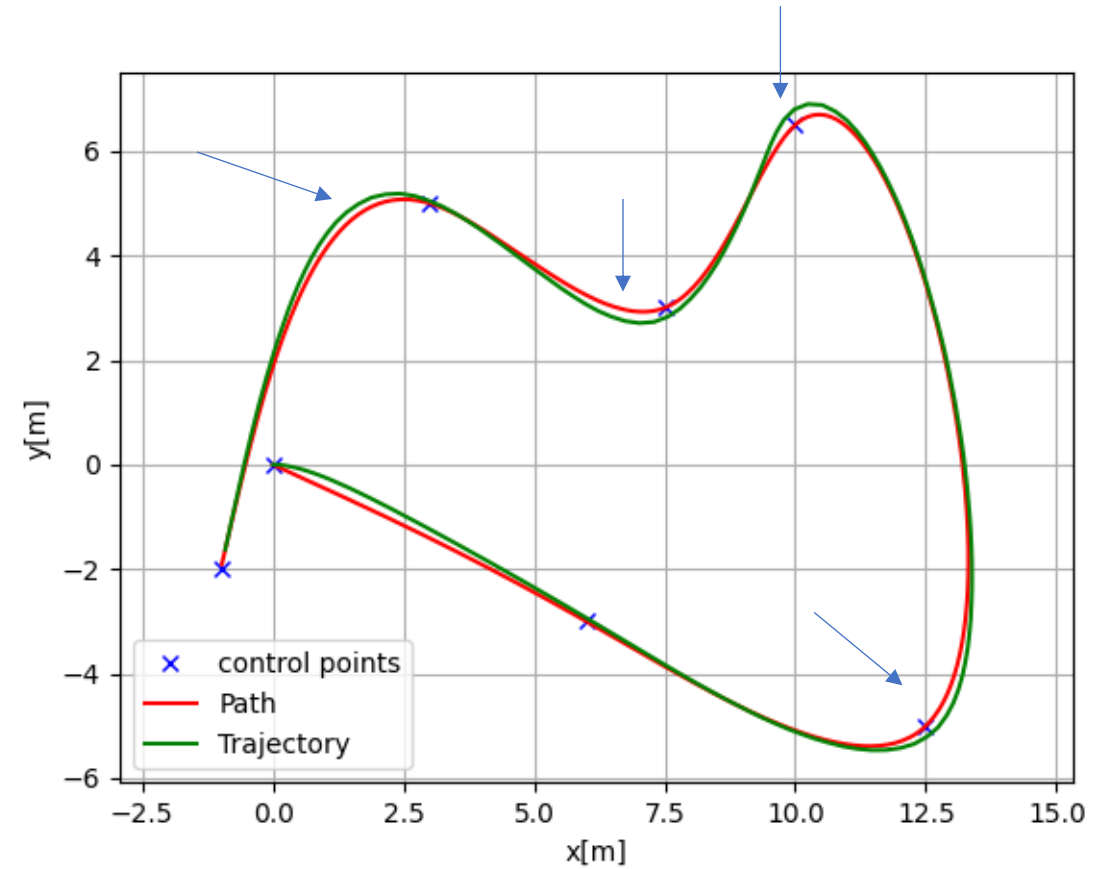
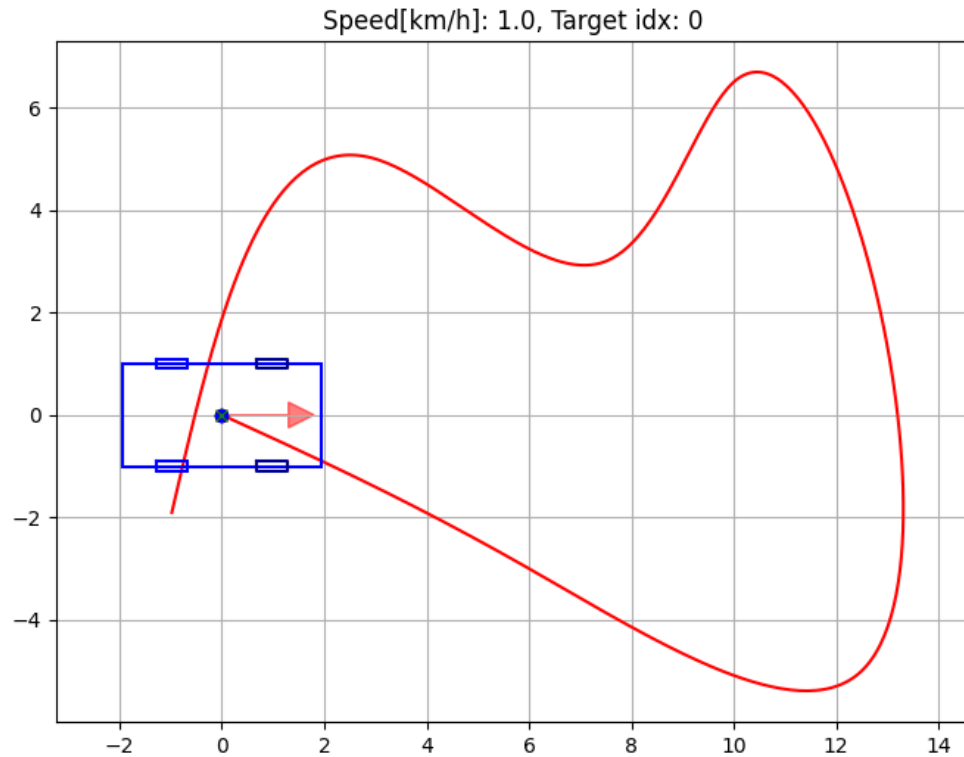
fb = pi_2_pi((-K @ x)[0, 0]) # 负反馈 (-Kx)



Part 4 | LQR steering control



LQR steering control



目录

Contents

01 课程内容安排

02 Pure pursuit tracking

03 Stanley control

04 LQR steering control

05 **LQR speed and steering control**

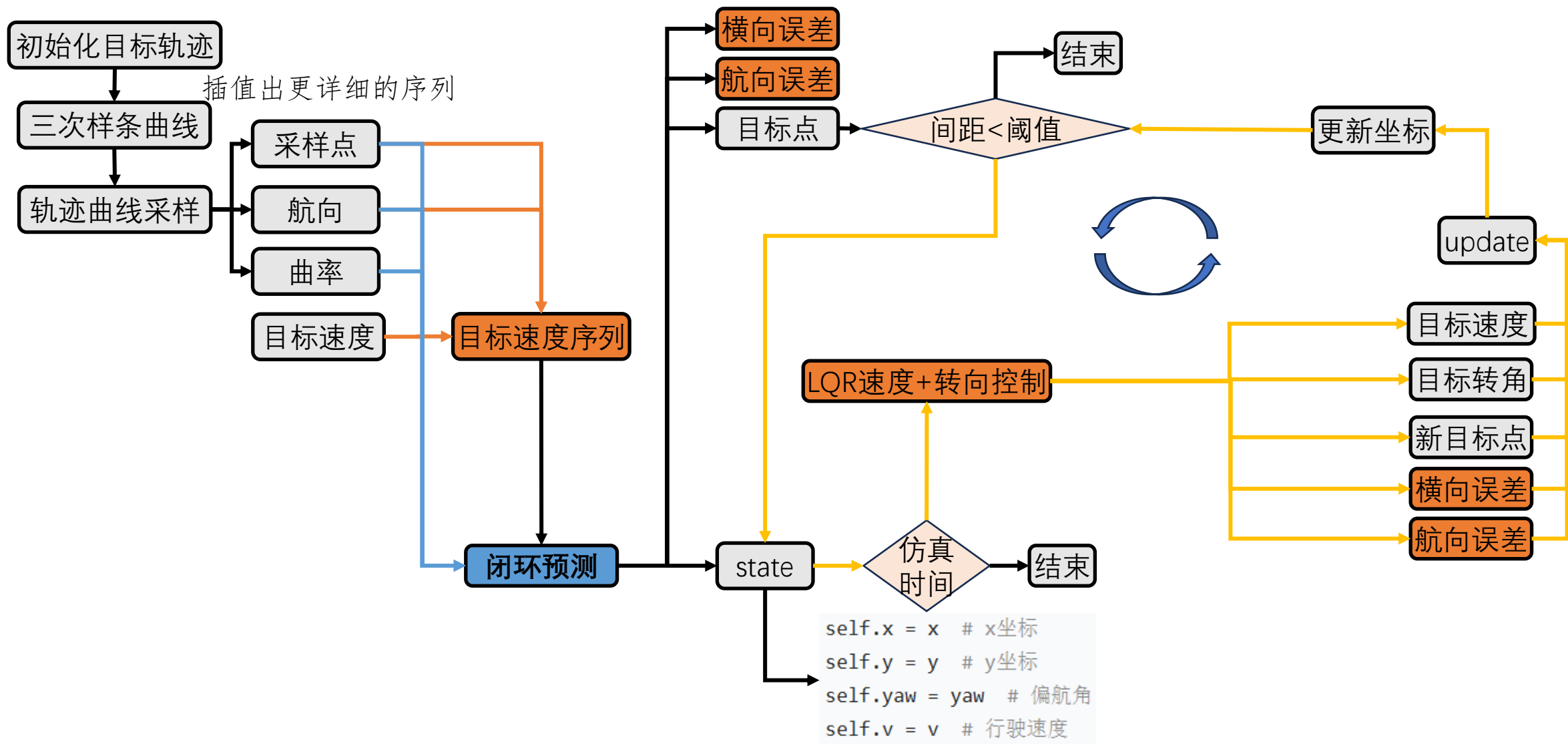
06 Model predictive speed and steering control

Part 5 | LQR speed and steering control



LQR speed and steering control

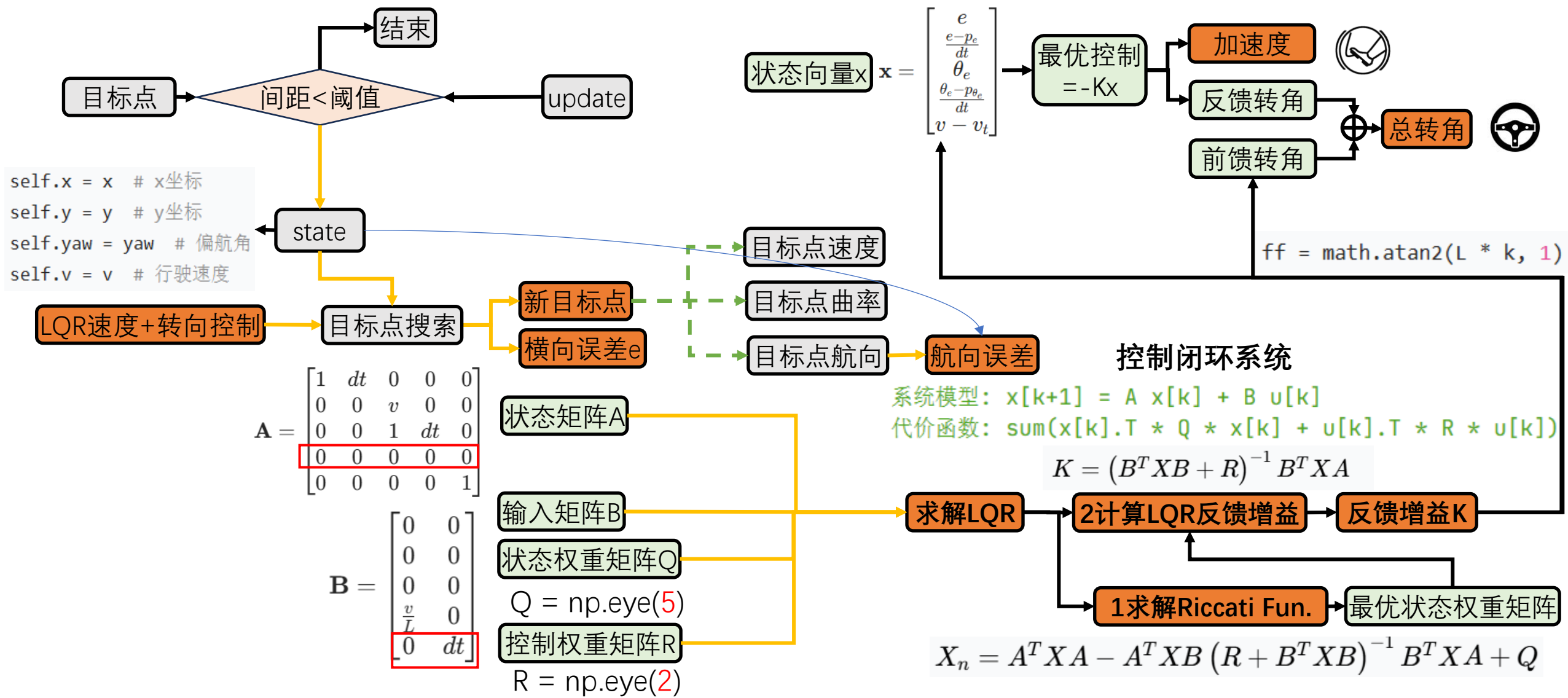
在横向控制的基础上，进一步添加LQR对速度的控制。



Part 5 | LQR speed and steering control

LQR speed and steering control

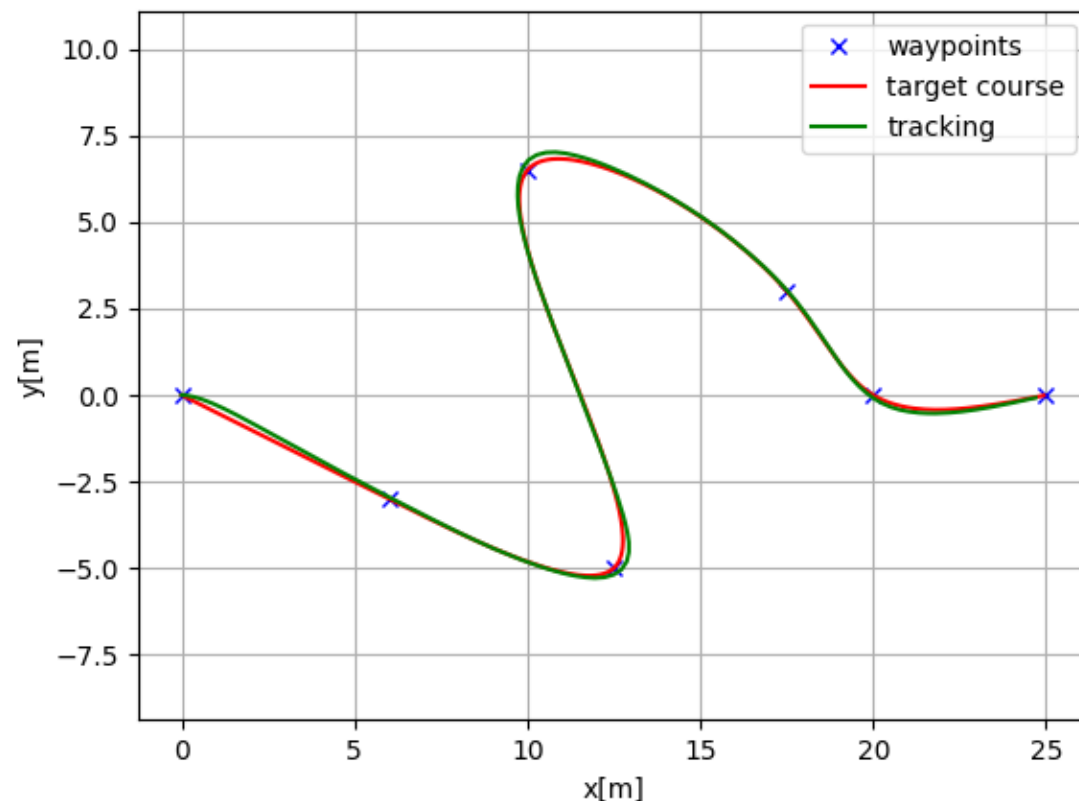
在横向控制的基础上，进一步添加LQR对速度的控制。



Part 5 | LQR speed and steering control



LQR speed and steering control



使用LQR统一框架进行协同优化，纵向和横向控制，对复杂急弯和高速运动控制更好。
但是矩阵计算维度增加。

实验要求： 1) 查询任意F1赛道，构建相应路径跟踪轨迹； 2) 对比LQR的横向控制器 和 LQR的纵横向控制器在你所构建的赛道中的表现结果。（使用不同颜色轨迹进行对比）； 3) 观察其它车辆动力学参数对模型影响。

目录

Contents

01 课程内容安排

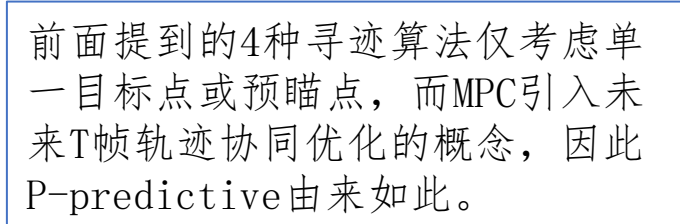
02 Pure pursuit tracking

03 Stanley control

04 LQR steering control

05 LQR speed and steering control

06 Model predictive speed and steering control



Part 6 | MPC

》 MPC

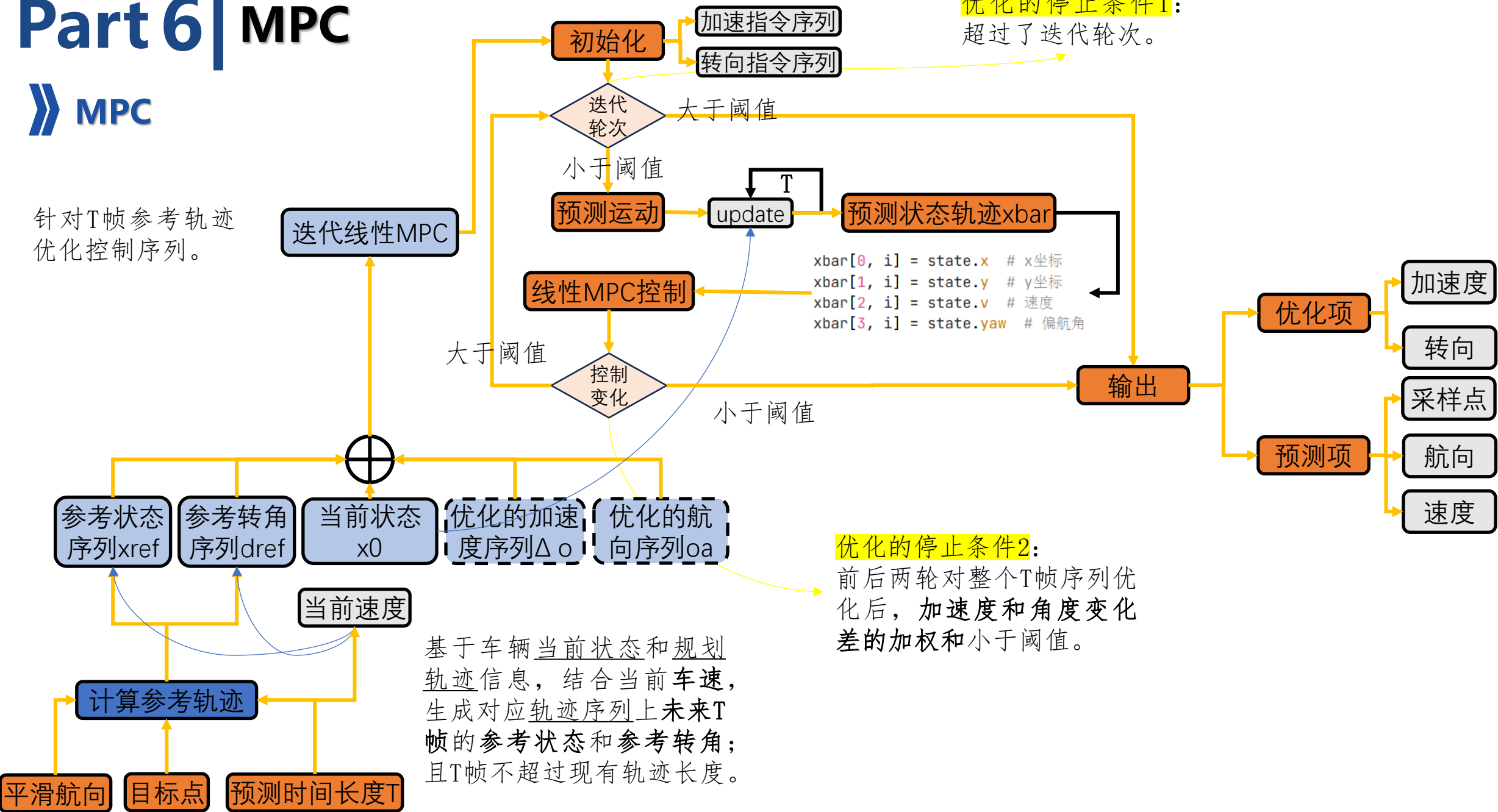
针对T帧参考轨迹
优化控制序列。

优化的停止条件1:
超过了迭代轮次。

```
xbar[0, i] = state.x # x坐标  
xbar[1, i] = state.y # y坐标  
xbar[2, i] = state.v # 速度  
xbar[3, i] = state.yaw # 偏航角
```

优化的停止条件2:
前后两轮对整个T帧序列优化后，
加速度和角度变化差的加权和小于阈值。

基于车辆当前状态和规划
轨迹信息，结合当前车速，
生成对应轨迹序列上未来T
帧的参考状态和参考转角；
且T帧不超过现有轨迹长度。



Part 6 | MPC

» MPC

参考状态
序列xref

参考转角
序列dref

当前状态
x0

预测状态
轨迹xbar

```
xbar[0, i] = state.x # x坐标  
xbar[1, i] = state.y # y坐标  
xbar[2, i] = state.v # 速度  
xbar[3, i] = state.yaw # 偏航角
```

线性MPC控制

控制输入代价矩阵
 $R = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix}$

控制输入变化量代价矩阵
 $R_d = \begin{bmatrix} 0.01 & 0 \\ 0 & 1.0 \end{bmatrix}$

状态跟踪代价矩阵
 $Q = \begin{bmatrix} 1.0 & 0 & 0 & 0 \\ 0 & 1.0 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0.5 \end{bmatrix}$

状态矩阵
 $x=4*(T+1)$

控制矩阵
 $u=2*T$

控制输入代价

累加二次型

$\text{cost} += \text{cvxpy.quad_form}(u[:, t], R)$

⊕

状态跟踪代价

累加二次型

$\text{cost} += \text{cvxpy.quad_form}(xref[:, t] - x[:, t], Q)$

⊕

控制输入变化代价

累加二次型

$\text{cost} += \text{cvxpy.quad_form}(u[:, t + 1] - u[:, t], R_d)$

⊕

最后一步状态跟踪代价

累加二次型

$\text{cost} += \text{cvxpy.quad_form}(xref[:, T] - x[:, T], Q_f)$

累计T帧

Part 6 | MPC



$$\mathbf{x}_k = [x_k \ y_k \ v_k \ \psi_k]^\top$$

$$\mathbf{u}_k = [a_k \ \delta_k]^\top$$

DT: 时间步长
WB: 车辆轴距

车辆运动学线性化模型:

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k + \mathbf{C}$$

状态转移矩阵A:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & DT \cdot \cos(\psi_k) & -DT \cdot v_k \cdot \sin(\psi_k) \\ 0 & 1 & DT \cdot \sin(\psi_k) & DT \cdot v_k \cdot \cos(\psi_k) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{DT \cdot \tan(\delta_k)}{WB} & 1 \end{bmatrix}$$

描述当前状态对下一状态的线性影响 (如位置随速度和偏航角的变化)

控制输入矩阵B:

$$\mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ DT & 0 \\ 0 & \frac{DT \cdot v_k}{WB \cdot \cos^2(\delta_k)} \end{bmatrix}$$

描述控制输入对状态的线性影响 (如加速度改变速度, 转向角改变偏航角)

常数项矩阵C:

$$\mathbf{C} = \begin{bmatrix} DT \cdot v_k \cdot \sin(\psi_k) \cdot \psi_k \\ -DT \cdot v_k \cdot \cos(\psi_k) \cdot \psi_k \\ 0 \\ -\frac{DT \cdot v_k \cdot \delta_k}{WB \cdot \cos^2(\delta_k)} \end{bmatrix}$$

补偿线性化过程中忽略的高阶项误差, 提高近似精度

参考状态
序列xref

参考转角
序列dref

当前状态
x0

预测状态
轨迹xbar

```
xbar[0, i] = state.x # x坐标  
xbar[1, i] = state.y # y坐标  
xbar[2, i] = state.v # 速度  
xbar[3, i] = state.yaw # 偏航角
```

Part 6 | MPC

» MPC

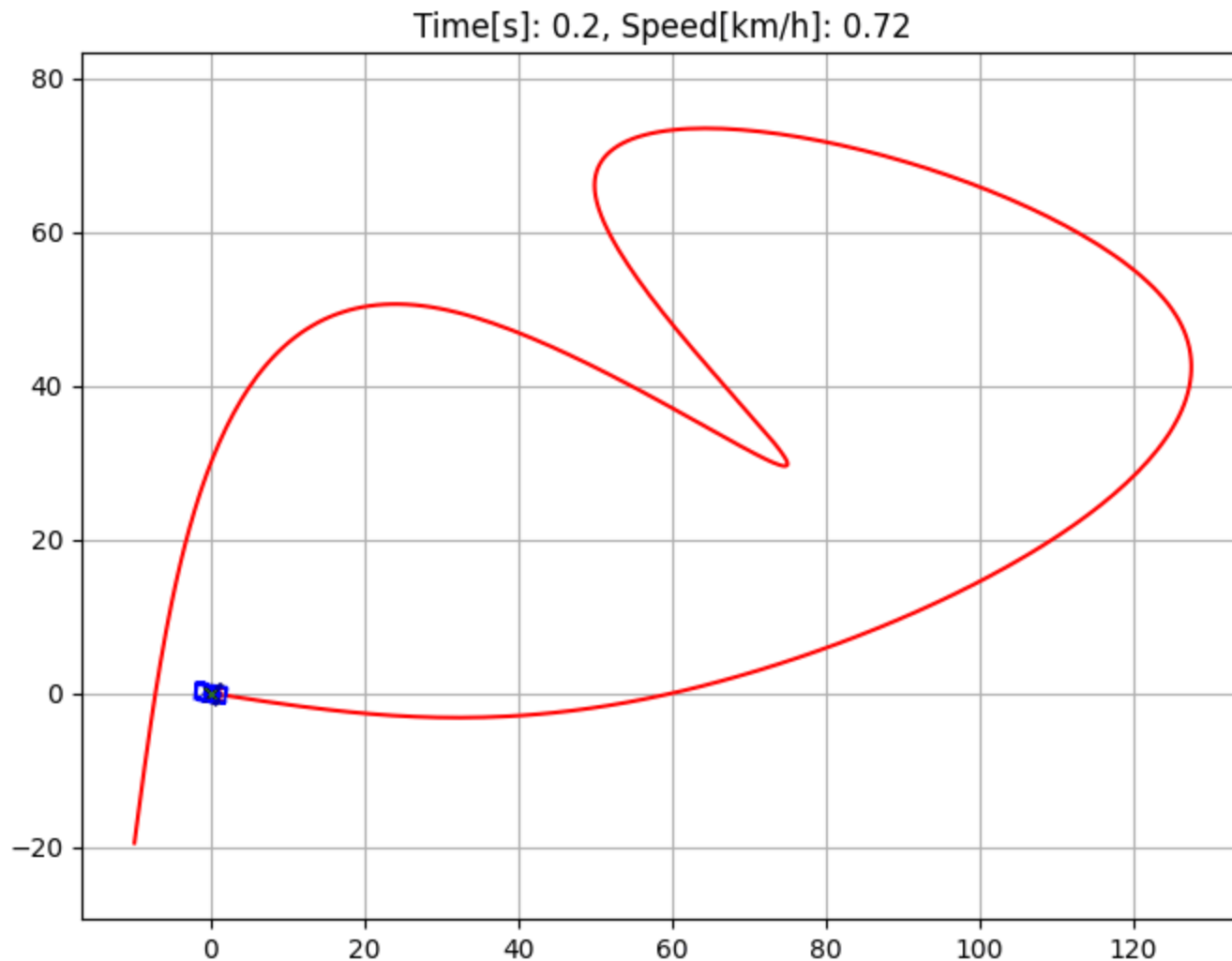
参考状态
序列xref

参考转角
序列dref

当前状态
x0

预测状态
轨迹xbar

```
xbar[0, i] = state.x # x坐标  
xbar[1, i] = state.y # y坐标  
xbar[2, i] = state.v # 速度  
xbar[3, i] = state.yaw # 偏航角
```



累计T帧

```
@ u[:, t] + C]  
t + 1] - u[1, t]) <=   
r]
```

ACCEL]

STEER]

} update

取控制指令的第一
辆运动，更新车辆
te)。

Part 6 | 最终实验

最终实验：

- 1) 五人一组为单位，依次选择科研1、科研2、科研3一层、体育馆地下停车场，构建地图。
- 2) 选择任意的 定位-建图-规划-寻迹算法进行组合，每人负责一个模块（寻迹模块负责车辆动力学建模），第5人负责地图制作、协同文档和模块集成测试。
 - 2.1) 车辆动力学模型须有对应实际车型，如小米su7 ultra等，相应建模参数可参考官网（提交报告需体现ref）。
- 3) 提交的代码，针对1构建的地图，支持选择任意起点和终点（除墙体等障碍物）进行模拟仿真，包含定位-建图-规划-寻迹四个模块的串行执行。
- 4) 每个模块具备量化评估输出。
- 5) 进行可视化输出，保存到mp4；撰写课程报告：包含四个模块的设计和单元测试、总体集成和集成测试，5个章节的内容（每人负责一个章节）。
- 6) 基于每组提交的内容课程作业，以及最终实验的成绩进行综合量化。

开始实验



北京航空航天大学
BEIHANG UNIVERSITY